

ARCMiS: Agentic Repository-Level Code Migration for Small Language Models

September 2026

Abstract

We present ARCMiS (Agentic Repository-level Code Migration for SLMs), a harness that migrates whole source codebases from one programming language to another using small language models (SLMs, 7B–27B parameters) served locally. The core method is a multi-agent system (MAS): a manager loop over a ten-phase state machine, delegating to ten specialist roles through parsed decision lines. Static agents meet dynamic selection—a task graph with dependency-gated fan-out, team collectives for per-batch translation, a deterministic tool guard on every call, per-role-class turn budgets, a stagnation breaker, and a fleet model ladder. All state lives on a character-capped shared blackboard. Long-context overflow is absorbed by snapcompact, which renders discarded history to PNG bitmap frames instead of calling an LLM summarizer. Scoring is toolchain-only: the target build and test commands must pass in the produced workspace, and agent self-reports never gate. In a 180-cell survey across three methods and three SLMs, only 5 of 180 runs produced a toolchain-verified migration, and all five came from the simplest method on the smallest project. A Meta-Harness-style autooptimise loop diagnosed the binding failure of the MAS baseline—reasoning-block truncation at the 4096-token output budget stalling every run at the Contract phase—and a one-line budget fix (`max_output_tokens 4096 → 8192`) let the first translated and toolchain-verified project land. We report the architecture, the honest negative results, and the failure taxonomy that disciplined the search.

1 Introduction

Repository-level code migration—translating an entire codebase from one language to another while keeping its behavior—is one of the hardest open tasks for coding agents. It needs three things at once: reading and understanding code that does not fit in one context window, planning a dependency-respecting translation order, and producing code that passes real toolchains. Large frontier models can attempt this with ad-hoc prompting; small language models cannot, and small models are the only affordable option for sustained local use: they run on one consumer GPU, cost nothing per token, and keep proprietary source on the machine.

ARCMiS targets exactly this regime. It migrates source codebases with 7B–27B models served locally through ollama, on a single RTX 3090. The design constraint that shapes the whole system is *honesty of scoring*: an agent’s claim of success counts for nothing. A run succeeds only when the target toolchain compiles the produced workspace and its test command exits zero. LLM-graded proxy scores, when recorded at all, never gate.

This paper makes four contributions:

- A MAS methodology for repository-level migration with SLMs: ten phases, ten specialist roles, a shared blackboard, and a deterministic guard (Section 3).

- Supporting infrastructure purpose-built for small local models: character-capped shared state, bitmap-frame context compaction, and hashline-based file invalidation (Section 4).
- Honest measurements: three method surveys (180 and 105 cells) and a Meta-Harness-style optimization loop with a diagnosed, mechanism-level fix (Sections 6 and 6.4).
- A failure taxonomy for SLM agents at this scale: reasoning-block truncation, provider 500s on continuation shapes, and stagnation (Section 7).

2 Related Work

Reasoning-and-acting agents. ReAct [19] interleaves chain-of-thought with tool actions and is the base loop for our monolith baseline. Reflexion [14] adds verbal self-reflection across retries; Self-Refine [9] iterates a generate–feedback–revise loop. ARCMiS borrows the iterate-on-failure shape but moves the feedback to deterministic judges and toolchain evidence, because 27B models self-assess poorly.

Multi-agent frameworks. MetaGPT [3] assigns a software company’s roles (product manager, architect, engineer) to LLM agents over standardized artifacts. AutoGen [15] builds applications from conversing agents with human-in-the-loop checkpoints. CAMEL [8] studies role-playing communicative agents. These frameworks assume strong models; ARCMiS keeps the role decomposition but replaces free-form conversation with a parsed decision protocol, a static registry, and a state machine, because conversation overhead is exactly what small turn budgets cannot afford.

Software-engineering agents. SWE-agent [18] shows that a well-designed agent–computer interface matters as much as the model; our hashline file tools and guard are the same principle at small scale. Agentless [16] demonstrates that a simple pipeline with localizer–repairer–validator stages beats complex agents on some tasks; our survey results agree in the small-project regime and our MAS adds dynamic scheduling for the parts that do not fit a fixed pipeline.

Tool use. Toolformer [13] teaches models to call APIs self-supervised; Gorilla [11] fine-tunes for API invocation. ARCMiS uses neither: the 17-tool surface is fixed, and the guard, not the model, enforces the calling discipline.

Code translation. TransCoder [4] does unsupervised transpilation with neural machine translation; AlphaTrans [1] is a neuro-symbolic repository-level translator with behavioral checks. Codamosa [6] escapes coverage plateaus by mixing LLM test generation with search-based testing. Our problem sets reuse the ReCodeAgent dataset’s project pairs, and our validator–tester loop applies the test-generation lesson with a per-module cap.

Small-model reasoning. DeepSeek-R1 [2] and Qwen3 [17] make deliberate reasoning (think blocks) standard on open models at our scale. This is a double-edged capability: long think blocks consume output-token budgets and directly caused the truncation failure we diagnose in Section 6.4. Speculative decoding [7] (and its MTP/SFT variants in the models we use) makes local inference fast enough for long agent runs, which is why the 100-agent-hour budgets of this paper fit on one GPU.

Harness optimization. Meta-Harness [10] optimizes an executable harness through an agentic outer loop where the proposer reads raw execution traces from the filesystem. ARCMiS adopts its experiment contract wholesale (Section 6.4); the ablation in that paper—raw traces beat summaries for the proposer—is what motivated our immutable, greppable experiment directories.

3 The ARCMiS Methodology

3.1 Overview

A run is one migration attempt against one problem set: a source codebase in language L_s and a target language L_t , with a test command defined in the config. The harness (a Rust binary built on rig 0.42 [19]) spawns a MAS whose members share one blackboard directory. The manager never writes product code; specialists never re-plan. Table 1 shows the role taxonomy.

Role	Class	Responsibility
Manager	control	Owens plan and task list; emits DECISION lines; no tools.
Analyst	worker	Reads the codebase; writes the source map and graphs.
Architect	worker	Writes the migration contract (analysis/brief.md).
Planner	worker	Produces dependency-ordered translation batches.
Translator	worker	Writes target-language modules for one batch.
Validator	judge	Judges one translated batch; re-runs the suite; read-only.
Tester	worker	Adds tests; runs the test command.
Failure-analyst	judge	Classifies one failure into a five-category taxonomy.
Critic	judge	End-of-run adversarial review; read-only.
Repairer	worker	Fixes one diagnosed failure.
Fleet-analyst	judge	Promotes or demotes roles along the model ladder.

Table 1: The role taxonomy. Manager is tier 1; specialists are tier 0; the translation collective is tier 2. Judge-class roles get small turn budgets; worker-class roles get large ones.

3.2 Phase state machine

The run walks ten phases: Preflight, Discovery, Contract, Planning, Pilot, Migration, Integration, Hardening, FinalValidation, Done. Forward transitions are earned: the manager’s **done** verb is accepted only when the phase’s exit condition holds *and* at least one delegation completed in the phase (the **phase_delegations** counter). Regressions are legal and recorded: Pilot and Migration may fall back to Discovery on a missing dependency, Integration to Migration, and so on. Anything else is a programming error, not a model decision. Each phase has a written exit condition in the manager prompt (for example: **Contract** requires **analysis/brief.md** to exist with a validator pass

covering it; `FinalValidation` requires the configured toolchain command to exit zero on the target workspace).

#	Phase	Exit evidence	Legal regressions
0	Preflight	deterministic checks, no model call	—
1	Discovery	source map + brief cite every module	—
2	Contract	brief.md + validator pass	—
3	Planning	every module in one batch; order compiles	—
4	Pilot	one batch translated, validated, tested	Pilot→Discovery
5	Migration	every batch translated and validated	Migration→Discovery
6	Integration	full test suite green	Integration→Migration
7	Hardening	critic verdict is pass	Hardening→Integration
8	FinalValidation	toolchain rerun exits zero	FinalVal.→Integration
9	Done	—	—

Table 2: The phase state machine. Forward moves are evidence-gated; regressions are recorded, not hidden.

3.3 Decision protocol and dynamic scheduling

The manager answers each round with plain text whose final lines carry **DECISION:** verbs—**delegate**, **replan**, **escalate**, **done**. Parsing text instead of expecting tool calls removes a whole failure class observed in the surveys: malformed tool-call JSON that the ollama daemon rejects with HTTP 500 (13% of all 220926 runs). The manager gets *no tools at all*; the round prompt carries the plan, notes, task list, ledger tail, and state, so the model cannot spend turns re-reading files (an observed pathology: 40 consecutive reads of **plan.md**).

Each **delegate** decision becomes a task node with optional **after:** dependencies and an optional **team** clause. The task graph executor runs the ready set (all dependencies done) with bounded fan-out (default 2). Because ollama serves one model at a time, fan-out interleaves turn streams rather than running parallel processes. Routing is a three-stage chain: the manager’s role hint, then a deterministic keyword pass over the task text, then (as a last resort) the router model. After each round the orchestrator re-scores the task list and the manager sees merged state, so a failing task can spawn a failure-analyst → repairer sub-chain without any phase regression.

Team collectives. A **team=true** delegation runs the translation collective translator → validator → tester, with failure-analyst → repairer re-entry, as an inner loop under one task id. The collective is tier 2, one level deep; the guard refuses deeper nesting. It has its own turn budget (**collective_turns**, default 40) and stops early on a validator pass.

3.4 Blackboard

All mutable state lives in a per-run directory written by the blackboard crate: **state.json** (phase, counters), **plan.md** and **notes.md** (character-capped at 4000 and 8000), **tasks.json** (task list mirror), **decisions.jsonl** and **failures.jsonl** (ledgers), and a dependency graph. Character caps are the discipline that keeps the shared state inside one read for a 16k-token context window; the writer truncates with an elision marker rather than silently growing the context of every later reader.

3.5 Guard

Every tool call from every agent passes one `on_tool_call` hook. The deterministic tier checks: role allowlist (from static tool metadata), path policy (`source/` is read-only; writes must stay inside the workspace), deny patterns for bash (`rm -rf`, `git push`, package installs, network commands, `sudo`), and per-delegation tool-call budgets. Rejections return `Skip(feedback)`: the model sees the reason and self-corrects, and the breaker counts repeats. Calls the deterministic tier marks ambiguous fall to an optional model-arbitration tier (the “Jev-as-guard” slot), deny-by-default while only one model exists. The guard is the single enforcement point; an earlier wrapper-object design (a `GuardedTool` adapter) was deleted because it was constructed but never on the call path.

3.6 Budgets, breaker, fleet ladder

Budgets are per role class: manager turns per round (default 20), worker turns per delegation (40), judge turns (40, kept small in spirit by prompting judges to promise few calls), collective turns (40), and a run-level round ceiling (60, with `max_rounds` often set to 20 in configs). A deterministic stagnation breaker stops the run after `stagnation_rounds` (3) consecutive rounds with no completed task, after repeated consecutive phase failures, or when a derived token estimate exceeds the run ceiling; it never consults a model. The fleet analyst owns a model ladder (weakest first, config-driven) and promotes a role one rung on repeated failure.

3.7 Scoring

A run succeeds only when the evaluator reruns the target toolchain (`cargo build + cargo test` for Rust targets, `pytest` for Python, syntax checking via `node --check` for JavaScript) in the produced workspace and every test passes. Agent self-reports are recorded but never trusted. This rule exists because the surveys caught real self-report traps: in the 220926 sweep, three ledger runs wrote passing per-problem records over empty workspaces; in the 2026-09-19 comparison, the harness compared a string `compilation_status == "pass"` while the schema asked the model for a boolean, so exit codes carried no signal until the scorer started re-running the toolchain itself.

4 Supporting Infrastructure

Hashline file tools. File tools (`read`, `write`, `edit`) address content through hashline tags (`PATH#TAG`). Edits name the tag they replace; a stale tag is an invalidation, not a silent overwrite. This gives the guard a path-shaped argument to check and gives the model a cheap merge-conflict signal.

Snapcompact. When the projected input of the next model call exceeds a per-role token threshold, the `snapcompact` hook keeps the most recent messages verbatim, serializes the discarded history to dense text, renders that text to PNG frames with a bundled public-domain 8x13 BDF bitmap font, and attaches the frames as image blocks. The pass is deterministic (same history in, same frames out) and costs no model call—unlike LLM summarization, which both spends budget and distorts. Per ADR 0022 the hook attaches to every role, with lower thresholds for the long-context specialists (translator, validator) that need it most.

Continuation patch. Ollama’s chat template for Qwen models rejects a message list that ends with tool results and carries no trailing user query (HTTP 500, “no user query found in messages”).

rig 0.42 produces exactly that shape on every multi-turn tool continuation. A completion-call hook appends a synthetic user message (“Tool results above. Continue the task from here.”) to the history; without it each rejected call burns a provider retry of 15–55 s.

Tool sandbox. Tool executions run against snapshot-checked workspaces, so a bad write can be rolled back without touching the source tree.

5 Experimental Setup

Model and hardware. All reported runs use `qwen3.8:27b-mtp-q4_K_M` (a 27B reasoning model with multi-token prediction, 4-bit quantized) served locally by ollama on one RTX 3090, fully offloaded. A 2B model (`minicpm5-2b:q8_0`) and a 27B fine-tune (`Swift-Qwen3.8-27B:dflash2`) appear only in the 220926 sweep. Typical generation settings: `num_ctx` 16384, temperature 0.2, `think: true`, one retry.

Problem sets. Two families. The *kata* family is the GildedRose-Refactoring-Kata (6.8 KB of C, translated to Rust) used for the method-comparison protocol. The *repository* family is the ReCodeAgent dataset: language-pair directories under `assets/ReCodeAgent/data/tool_projects` (c→rust, python→javascript, go→rust, java→python), with per-project configs. The autooptimise cycle-1 set spans six pairs from 125 to ~28k source LOC (Table 3).

Problem set	Source	Target	Source LOC
crust/nandc	C	Rust	125
skel/colorsyz	Python	JavaScript	355
crust/utf8	C	Rust	806
crust/libm17	C	Rust	7,009
oxidizer/stats	Go	Rust	5,625
alphatrans/commons-validator	Java	Python	28,110

Table 3: The cycle-1 autooptimise problem sets (measured on source trees; headers excluded for C). Test commands come from the config: `cargo test`, `pytest`, or `node --check`.

Budgets. The MAS configs fix `max_rounds` 20, manager turns 20 per round, worker turns 40 per delegation, judge turns 40, fanout 2, stagnation breaker at 3 rounds, `plan_cap` 4000, `notes_cap` 8000. Baselines used `max_output_tokens` 4096; the H1 candidate raises it to 8192 (Section 6.4).

Protocol. One process per cell, strictly sequential, fresh workspace per run, model offloaded between runs and verified by polling the daemon. Every run leaves an immutable experiment directory per the ADR 0020 contract: `manifest.json` (model, budgets, git revision, parents, hypothesis), config, `traces/turns.jsonl` (one line per model call, tool call, and tool result), the produced `workspace/`, captured streams, and a `result/` aggregate plus per-problem records with a staged failure taxonomy (`setup`, `analyze`, `plan`, `translate`, `validate`, `evaluate`).

6 Results

6.1 Method comparison on the kata (2026-09-19)

Three paper-faithful methods on the GildedRose C→Rust kata, 5 passes each, toolchain-only scoring (Table 4).

Method	Compiled	Full pass	Test pass rate	Wall
Monolith (ReAct + validator)	5/5	1/5	0.50 ± 0.00	97.4 ± 34.0 s
Ledger [5]	5/5	1/5	0.60 ± 0.22	116.3 ± 21.6 s
ReCodeAgent [12]	5/5	1/5	0.65 ± 0.22	160.5 ± 19.1 s

Table 4: Kata benchmark, revision 2 (paper-faithful implementations, 5 passes per method). The two multi-agent methods sit above the monolith (0.65, 0.60 vs. 0.50), but the deltas sit inside one SD; the evidence for scaffolds paying is one full pass in five for each method. An earlier flattened-implementation generation had the monolith on top (0.75); the v1 and v2 monolith columns are two harness generations, not one method twice.

Two readings carry forward. First, on a small task with a 27B model, the single agent is already at or above the regime where decomposition pays, and orchestration turns subtract from the fix budget—the papers’ own conclusion (scaffolds pay most where the model is weakest) predicts exactly this inversion. Second, wall time ranks stably with agent count: more agents, more seconds.

6.2 Survey 220926: capability dominates methodology (180 cells)

Grid: 4 projects (233–1110 C LOC) $\times$ 3 methods $\times$ 3 models $\times$ 5 reps. Toolchain-verified end-to-end success: 5 of 180 runs (2.8%), all on the 233-LOC project, all monolith, all on a 27B model. The 2B model never produced a compiling workspace. Ledger wrote 3 per-problem records, but every one was a manager self-report over an empty workspace—caught by post-sweep inspection, and the direct motivation for toolchain-only scoring. Recode never produced a record. The failure taxonomy (Table 5) put turn-budget exhaustion at 42% and daemon-rejected tool-call arguments at 13%.

Failure class	Count	Share
MaxTurns (budget exhausted mid-translation)	75	42%
Other (model-side protocol errors, empty streams)	70	39%
Invalid tool-call arguments (daemon HTTP 500)	23	13%
JSON parse of model output	4	2%
Toolchain-verified success	5	3%
Self-reported “pass” over empty workspace	3	2%

Table 5: Failure taxonomy over 180 runs (survey 220926). The multi-agent scaffolds divide the same turn budget across more workers, so they exhaust it sooner relative to useful work.

6.3 Survey 230926: budgets unbind, size binds (105 cells)

With generous budgets (120 turns, 8192 output tokens, 2-hour timeouts), MaxTurnsError went from 42% of runs to zero; mean turns used stayed under 16 per method. On 7 projects (185–3087 C LOC) $\times$ 3 methods $\times$ 5 reps with qwen3.8-27B, 39 of 105 cells (37%) verified. Success concentrated by

project size, not method: the 185–324 LOC band converted at 87–93% per attempt; the 635–3087 LOC band converted at 0% (nothing compiled), with a sharp transition between 324 and 635 LOC. Method differences were second-order and project-dependent: ledger was the only method to pass `chtrie` (5/5), monolith the only one to pass `murmurhash_c` (5/5), recode never won a cell outright but still passed 4/5 on two projects. Ledger used the fewest turns (mean 5.3) but the longest walls (mean 630 s); monolith used mean 11.8 turns and 316 s. The 220926 ledger self-report trap did not recur: all 15 ledger scored runs carried real evaluator steps.

6.4 The frontier loop: baselines stall at Contract

The autooptimise loop (ADR 0020, after Meta-Harness [10]) runs one immutable experiment directory per candidate, with the proposer reading raw traces, keeping a Pareto frontier over (pass rate, tests passed, tests failed), and never editing `scripts/` or tuning against the problem set.

Baseline stagnation. Every baseline MAS run across six problem sets aborted identically: 3 consecutive stalled rounds, final phase `Discovery` or `Contract`, no translated project. The colorsys baseline (20260926T213500Z) reached `Contract` and had its brief rejected (43 lines against a 40-line cap); the fix delegation then burned its entire 4096-token output budget on a single think-only turn (`finish_reason=Length`, empty text, no tool call), the provider raised “produced no answer,” the round stalled, and the breaker tripped. The crust-nandc baseline showed the identical signature. The cross-run invariant: *every* delegation that died did so with output tokens exactly equal to the 4096 cap and empty visible text—the reasoning block consumed the whole budget.

H1 diagnosis and fix. The mechanism chain: the config enables `think: true`; the 27B model emits long reasoning (`think`) blocks; rig counts them against `max_output_tokens`; and a turn that delivers no answer and is truncated (`finish_reason=Length`) is a hard completion error, not a retry with more budget. The harness treats the delegation as failed, the manager retries the same shape, the model re-thinks just as long, and three stalls trip the breaker. H1 proposed the minimal mechanism-targeted change: raise `max_output_tokens` 4096 → 8192 (the library default; the configs had pinned 4096 explicitly). Alternatives considered and deferred: `think: false` (kills deliberate reasoning), a continuation-on-length hook (needs provider internals), and loosening the validator’s line cap (symptom-level, against the leakage rules).

Effect. The H1 candidate on crust/nandc (20260927T112000Z, parents = the two nandc baselines, hypothesis recorded in the manifest) completed 20 rounds, 13 delegations, 12 of 13 tasks done, and reached `Hardening`—the first MAS run to leave the `Contract` phase—and its workspace passed the toolchain rerun with 2 tests passed, 0 failed. This is the first translated, toolchain-verified repository migration produced by the MAS method. It remains incomplete (the run ended at `Hardening`, not `Done`), and the other five H1 candidates in cycle 1 still stalled at `Discovery` or `Contract`; the fix unblocked the mechanism, not every run. H2 (queued for cycle 2) targets the second observed mechanism: a first-call request shape with an empty prompt field that ollama’s server-side chat template rejects with HTTP 500, which the continuation hook does not yet cover; 23 of 28 model calls in the base-crust-libm17 trace carried the empty prompt.

Frontier discipline. The frontier keeps non-dominated candidates rather than a champion: the H1-nandc candidate dominates the nandc baselines on pass rate and tests passed; the other five sets have no scored candidate yet, so their baselines stay on the frontier by default. Lineage lives in the manifests (`parents`), and every hypothesis is one causal sentence recorded before the run.

7 Discussion

Failure modes. Three failure modes dominate everything we have measured. (1) *Think-truncation*: reasoning blocks consume the output budget; the turn dies with `finish_reason=Length` and no visible answer; retries reproduce it deterministically at temperature 0.2. Fixed by budget, not by prompt surgery. (2) *Provider shape rejections*: ollama’s server-side chat template 500s on message lists without a trailing user query, both on tool continuations (patched by the hook) and on a first-call shape (H2, open). (3) *Stagnation*: without a breaker, the manager retries the same delegation shape indefinitely; the 3-round breaker converts an unbounded loop into a diagnosable artifact. A fourth mode, malformed tool-call JSON (13% of survey 220926), largely disappeared from the MAS runs after decisions moved from tool calls to parsed text.

Limitations. The verified-success frontier for a 27B model on C→Rust sits near 300–350 source LOC for single-shot methods; MAS pushed one 125-LOC repository through Contract to Hardening but no run has yet landed Done end-to-end. Fan-out is turn-level interleaving, not parallelism, while the fleet has one model slot. Autooptimise has completed one proposal–evaluate cycle; the leakage rule (holdout problem sets must differ from search sets) blocks any generalization claim until a second family is scored. The alphasrings set (28k LOC Java) exceeds what a 16k context can even inventory; it stands as a stress upper bound, not a current target.

Threats to validity. Sample sizes are small ($n=5$ per cell in the kata benchmark; single completions per problem set in the frontier loop), so no significance claims are made; deltas inside one SD are reported as such. One model and one hardware configuration dominate the data; the 220926 model-axis evidence is three models, one of which never worked. Infra failures contaminate raw counts (57 of 109 on-disk 230926 attempts were preflight aborts against a dead daemon) and are excluded from per-method rates, which the experiment contract now prevents by gating on daemon liveness. Finally, toolchain-only scoring can still pass a workspace whose tests are weaker than the source suite; the tester’s per-module generated-test cap (4) bounds but does not eliminate that gap.

8 Conclusion

ARCMiS shows that repository-level code migration with small local models is tractable only when the harness carries most of the discipline: evidence- gated phases, a deterministic guard on every tool call, character-capped shared state, toolchain-only scoring, and a stagnation breaker that turns silent loops into artifacts. The measured record is deliberately honest: the simplest method won the first two surveys, capability dominated methodology at the 2B/27B boundary, project size between 324 and 635 LOC marked a sharp capability wall, and the MAS method’s first verified migration came from a one-line output-budget fix that the optimization loop found by reading raw traces—not from a cleverer prompt. The infrastructure (the experiment contract, the frontier discipline, the failure taxonomy) is the durable contribution; it makes every next hypothesis cheap to test and every claimed success impossible to fake.

References

- [1] Anirudh Arya, Saikat Chakraborty, Shraddha Negi, Aditya Desai, et al. AlphaTrans: A neuro-symbolic approach to translate legacy code. *arXiv preprint arXiv:2411.11018*, 2024.

- [2] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [4] Marie-Anne Lachaux, Baptiste Rozière, Marc Goyal, Guillaume Lozano, and Guillaume Lample. Unsupervised translation of programming languages. *arXiv preprint arXiv:2006.03511*, 2020.
- [5] Ledger authors. Ledger: Manager-worker LLM agents with a shared task ledger. *arXiv preprint arXiv:2608.26480*, 2026.
- [6] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. *arXiv preprint arXiv:2305.01810*, 2023.
- [7] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *arXiv preprint arXiv:2211.17192*, 2023.
- [8] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. *arXiv preprint arXiv:2303.17760*, 2023.
- [9] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegraffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.
- [10] Meta-Harness authors. Meta-harness: Agentic optimization of execution harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- [11] Shishir G Patil, Tianjun Guo, Ruochen Cheng, Maxwell Ligocki, et al. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.
- [12] ReCodeAgent authors. ReCodeAgent: A pipeline agent for repository-level code migration. *arXiv preprint arXiv:2604.07341*, 2026.
- [13] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- [14] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [15] Qingyun Wu, Gagan Bansal, Jieyuan Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [16] Chunqiu Steven Xia, Yuxiang Deng, Soren Dun, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

- [17] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [18] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [19] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.